

UNIVERSIDAD DE MONTERREY

Escuela de Ingeniería y Tecnología

Ingeniería en Tecnologías Computacionales

Diseñar Integridad y Reglas de negocio en PostgreSQL

Autor(es)

600046 Fernando Olivares Del Valle

Profesor (a)

Prof. Raúl Morales Salcedo

Doy mi palabra que he realizado esta actividad con integridad académica

San Pedro Garza García, N.L. 27 de Agosto, 2026

Diseñar Integridad y Reglas de negocio en PostgreSQL

Catálogos con nombre único

```
SQL
CREATE TABLE formatos (
    id_formato      SERIAL PRIMARY KEY,
    nombre          VARCHAR(50) NOT NULL UNIQUE -- Ej: Tapa dura, Tapa blanda,
Digital, Audiolibro
);

CREATE TABLE generos (
    id_genero       SERIAL PRIMARY KEY,
    nombre          VARCHAR(50) NOT NULL UNIQUE -- catálogo de
géneros/categorías
);

CREATE TABLE conceptos (
    id_concepto     SERIAL PRIMARY KEY,
    nombre          VARCHAR(150) NOT NULL UNIQUE -- ej: "Realismo mágico"
);
```

Restricción:

- **Primary key:** Clave incremental que permite identificar y referencias el catálogo desde la tabla de libros.
- **Unique:** Evita duplicar el mismo nombre de formato/género/concepto con distintos ID
- **NOT NULL:** Un catálogo no puede tener un nombre vacío

Autores

```
SQL
CREATE TABLE autores (
    id_autor        SERIAL PRIMARY KEY,
    nombre          VARCHAR(150) NOT NULL,
    nacionalidad     VARCHAR(80)
);
```

Restricción:

- **Primary key (id_autor):** Clave sustituta ya que el nombre del autor no se puede utilizar como clave porque pueden existir duplicados.
- **Sin UNIQUE en nombre:** Los nombres si se pueden repetir entre los autores.
- **nombre NOT NULL:** Un autor no puede no tener nombre.
- **nacionalidad sin NOT NULL:** Es información complementaria y se puede desconocer en el momento.

Libros

SQL

```
CREATE TABLE libros (  
    isbn          VARCHAR(13) PRIMARY KEY,  
    titulo        VARCHAR(255) NOT NULL,  
    anio_publicacion SMALLINT NOT NULL CHECK (anio_publicacion > 0),  
    precio        NUMERIC(10,2) NOT NULL CHECK (precio >= 0),  
    stock         INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0),  
    id_formato    INTEGER NOT NULL REFERENCES formatos(id_formato),  
    fecha_creacion TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

Restricción:

- **Primary key (isbn):** Identificador único y natural para cada uno de los libros. No es necesario que exista un id.
- **CHECK (año_publicación):** Los años de publicación no pueden ser inválidos (negativos o cero) para evitar errores de captura.
- **CHECK (precio):** Un precio no puede ser negativo, esto para evitar errores de entrada
- **CHECK (stock):** Evita que el inventario quede en negativo, para el manejo de errores.
- **id_formato NOT NULL REFERENCES:** Todos los libros tiene que tener un formato válido y existente.

Usuarios

SQL

```
CREATE TABLE usuarios (  
  id_usuario      SERIAL PRIMARY KEY,  
  nombre          VARCHAR(150) NOT NULL,  
  correo          VARCHAR(150) NOT NULL UNIQUE,  
  password_hash   VARCHAR(255) NOT NULL,  
  es_admin        BOOLEAN NOT NULL DEFAULT FALSE,  
  fecha_registro  TIMESTAMP NOT NULL DEFAULT NOW(),  
  activo          BOOLEAN NOT NULL DEFAULT TRUE  
);  
  
CREATE UNIQUE INDEX un_solo_admin  
  ON usuarios (es_admin)  
  WHERE es_admin = TRUE;
```

Restricción:

- **Primary key (id_usuario):** Identificador para establecer sesiones y relaciones
- **UNIQUE (correo):** Un usuario solamente puede tener un correo como identificador para el login.
- **password_hash (NOT NULL):** Toda cuenta debe tener una credencial almacenada para autenticarse
- **es_admin DEFAULT FALSE:** Ningún usuario nuevo es administrador, se debe de otorgar el rol.
- **un_solo_admin:** Índice único para identificar al administrador y que impide tener más de uno simultáneamente.

libro_autor / libro_género

SQL

```
-- Libro <--> Autor  
CREATE TABLE libro_autor (  
  isbn           VARCHAR(13) REFERENCES libros(isbn) ON DELETE CASCADE,  
  id_autor       INTEGER REFERENCES autores(id_autor) ON DELETE RESTRICT,  
  PRIMARY KEY (isbn, id_autor)  
);  
  
-- Libro <--> Género  
CREATE TABLE libro_genero (  
  isbn           VARCHAR(13) REFERENCES libros(isbn) ON DELETE CASCADE,
```

```
id_genero    INTEGER REFERENCES generos(id_genero) ON DELETE RESTRICT,
PRIMARY KEY (isbn, id_genero)
);
```

Restricción:

- **Primary key (isbn y id_autor):** Impide asociar el mismo autor dos veces al mismo libro.
- **Primary key (isbn y id_genero):** Impide asociar el mismo género dos veces al mismo libro.
- **FK isbn REFERENCES libros ON DELETE CASCADE:** Si se elimina un libro, se eliminarán las asociaciones con autores. En el segundo caso, también se eliminarán los géneros.
- **FK id_author REFERENCES autores ON DELETE RESTRICT:** No se puede eliminar un autor que todavía tiene libros asociados, esta operación se rechaza.
- **FK id_genero REFERENCES generos ON DELETE RESTRICT:** No se puede eliminar un genero que todavía tiene libros asociados, esta operación se rechaza.

libro_concepto

SQL

```
CREATE TABLE libro_concepto (
    isbn          VARCHAR(13) REFERENCES libros(isbn) ON DELETE CASCADE,
    id_concepto   INTEGER REFERENCES conceptos(id_concepto) ON DELETE RESTRICT,
    definicion    TEXT NOT NULL,
    PRIMARY KEY (isbn, id_concepto)
);
```

Restricción:

- **Primary key (isbn y id_concepto):** Un concepto no puede repetirse con el mismo libro.
- **Definicion NOT NULL:** Un concepto no puede no tener una definición asociada, este campo tiene que estar completo.
- **ON DELETE CASCADE (isbn)/ ON DELETE RESTRICT (Id_concepto):** Si el libro se borra, de igual manera el concepto se borrará en la relación

imagenes

SQL

```
CREATE TABLE imagenes (  
  id_imagen    SERIAL PRIMARY KEY,  
  isbn         VARCHAR(13) NOT NULL REFERENCES libros(isbn) ON DELETE  
CASCADE,  
  url         VARCHAR(500) NOT NULL,  
  es_principal BOOLEAN NOT NULL DEFAULT FALSE,  
  orden       SMALLINT DEFAULT 0  
);  
  
CREATE UNIQUE INDEX una_imagen_principal  
  ON imagenes (isbn)  
  WHERE es_principal = TRUE;
```

Restricción:

- **Primary key (id_imagen):** A cada imagen le corresponde solamente un id para poder identificarlo.
- **isbn NOT NULL REFERENCES libros ON DELETE CASCADE:** Todas las imagenes tienen que tener un libro, si este libro se elimina sus imágenes también deberán de hacerlo.
- **ON DELETE CASCADE (isbn)/ ON DELETE RESTRICT (Id_concepto):** Si el libro se borra, de igual manera el concepto se borrará en la relación.
- **una_imagen_principal:** Índice único para identificar al libro, que garantiza como máximo una imagen por libro, marcado con un booleano.

Pruebas negativas de integridad:

SQL

#isbn duplicado

```
INSERT INTO libros (isbn, titulo, anio_publicacion, precio, stock, id_formato)
VALUES ('9780307474728', 'Libro duplicado', 2020, 100.00, 5, 1);
```

 fodv004@maquina-01:/opt/udem/WebMonolitico

```
library=> INSERT INTO libros (isbn, titulo, anio_publicacion, precio, stock, id_formato)
VALUES ('9780000000001', 'Libro duplicado', 2020, 100.00, 5, 1);
ERROR:  duplicate key value violates unique constraint "libros_pkey"
DETAIL:  Key (isbn)=(9780000000001) already exists.
library=>
```

SQL

#stock negativo

```
INSERT INTO libros (isbn, titulo, anio_publicacion, precio, stock, id_formato)
VALUES ('999999999999', 'Libro stock malo', 2020, 100.00, -5, 1);
```

 fodv004@maquina-01:/opt/udem/WebMonolitico

```
library=> INSERT INTO libros (isbn, titulo, anio_publicacion, precio, stock, id_formato)
VALUES ('999999999999', 'Libro stock malo', 2020, 100.00, -5, 1);
ERROR:  new row for relation "libros" violates check constraint "libros_stock_check"
DETAIL:  Failing row contains (999999999999, Libro stock malo, 2020, 100.00, -5, 1, 2026-08-30 16:08:19.496979).
library=>
```

SQL

#precio inválido (negativo)

```
INSERT INTO libros (isbn, titulo, anio_publicacion, precio, stock, id_formato)
VALUES ('9999999999998', 'Libro precio malo', 2020, -50.00, 5, 1);
```

fodv004@maquina-01:/opt/udem/WebMonolitico

```
library=> INSERT INTO libros (isbn, titulo, anio_publicacion, precio, stock, id_formato)
VALUES ('9999999999998', 'Libro precio malo', 2020, -50.00, 5, 1);
ERROR: new row for relation "libros" violates check constraint "libros_precio_check"
DETAIL: Failing row contains (9999999999998, Libro precio malo, 2020, -50.00, 5, 1, 2026-08-30 16:09:00.244314).
library=>
```

SQL

#FK inexistente

```
INSERT INTO libros (isbn, titulo, anio_publicacion, precio, stock, id_formato)
VALUES ('9999999999997', 'Libro formato malo', 2020, 100.00, 5, 999);
```

fodv004@maquina-01:/opt/udem/WebMonolitico

```
library=> INSERT INTO libros (isbn, titulo, anio_publicacion, precio, stock, id_formato)
VALUES ('9999999999997', 'Libro formato malo', 2020, 100.00, 5, 999);
ERROR: insert or update on table "libros" violates foreign key constraint "libros_id_formato_fkey"
DETAIL: Key (id_formato)=(999) is not present in table "formatos".
library=>
```


SQL

#Eliminación que viole una relación

```
DELETE FROM autores WHERE id_autor = 1;
```

```
fodv004@maquina-01:/opt/udem/WebMonolitico
library=> DELETE FROM autores WHERE id_autor = 1;
ERROR: update or delete on table "autores" violates foreign key constraint "libro_autor_id_autor_fkey" on table "libro_autor"
DETAIL: Key (id_autor)=(1) is still referenced from table "libro_autor".
library=>
```

SQL

#Creación de un segundo administrador

```
INSERT INTO usuarios (nombre, correo, password_hash, es_admin)
VALUES ('Segundo Admin', 'otro@admin.com', 'hash_falso', TRUE);
```

```
fodv004@maquina-01:/opt/udem/WebMonolitico
library=> INSERT INTO usuarios (nombre, correo, password_hash, es_admin)
VALUES ('Segundo Admin', 'otro@admin.com', 'hash_falso', TRUE);
ERROR: duplicate key value violates unique constraint "un_solo_admin"
DETAIL: Key (es_admin)=(t) already exists.
library=>
```